PROGRAMMING IN JAVA

ASSIGNMENT 3 – CO3

Exception Handling and Multithreading

Name: J. Subhaashree

Register Number: 192424008

Course: Programming in Java

Course Outcome: CO3

Total Marks: 50

Contents

1. Executive Summary

2. Q1 – Handling Failure in a Bank ATM Simulation

3. Q2 – User-Defined Exceptions and Propagation in an Online Examination Portal

4. Q3 – Resource Safety, Debugging and Robust Code

5. Q4 – Thread Creation and the Thread Life Cycle

6. Q5 – Race Conditions in a Cinema Seat-Booking System

7. Q6 – Deadlock and Inter-Thread Communication

8. Q7 – Design a Concurrent, Fault-Tolerant Processor from First Principles

9. Overall Conclusion

Individual Contribution Record

Question	Individual responsibility
Q1	Exception hierarchy, failure classification, catch ordering, finally behaviour and tests
Q2	Custom exception hierarchy, propagation, checked/unchecked choice and chaining
Q3	Resource lifecycle, manual-close failure, try-with-resources and suppressed exceptions
Q4	Thread and Runnable, lifecycle, ten-run completion-order experiment and join
Q5	Race-condition analysis, synchronized booking and repeated tests
Q6	Deadlock analysis, lock ordering and producer-consumer implementation
Q7	ExecutorService design, fault injection, result collection and pool-size measurements

Executive Summary

This assignment implements and analyses seven Java tasks covering exception handling, resource safety, thread lifecycle, synchronization, inter-thread communication, deadlock prevention and concurrent fault-tolerant processing.

The submitted Colab execution exercised five ATM paths; preserved an IOException as the cause of a custom checked exception; compared manual resource handling with try-with-resources; recorded ten different thread completion orders; verified exactly ten cinema seats sold; collected tokens with a total of 210; and processed 10,000 score files with 9,800 successes and 200 failures.

Measured Q7 times were 605 ms (1 worker), 211 ms (2), 211 ms (4) and 163 ms (8). These are results from the submitted Colab run and depend on its runtime environment.

Q1 – Handling Failure in a Bank ATM Simulation

The ATM reads an account identifier, amount text and card-reader availability. Integer.parseInt validates numeric input. The program requires a positive amount and rejects withdrawals greater than the current balance. A missing card-reader file is translated from FileNotFoundException into a checked CardReaderException. The finally block records cleanup on successful and exceptional paths.

Failure classification: NumberFormatException is unchecked and occurs for non-numeric input. IllegalArgumentException is unchecked and represents a non-positive amount. IllegalStateException is unchecked and represents insufficient balance. FileNotFoundException is checked and is wrapped by the checked custom CardReaderException. JVM Errors such as OutOfMemoryError are not normal business failures and should not be caught as routine input errors.

Catch ordering: catch clauses must be ordered from specific to general. A broad catch(Exception e) before a narrower catch(NumberFormatException e) makes the latter unreachable. NumberFormatException extends IllegalArgumentException, so they cannot appear together as alternatives in one multi-catch. The submitted implementation handles the distinct validation exceptions and custom checked exception at the caller.

Test outcomes: T1 withdraws 500 successfully and leaves 4500. T2 supplies abc and produces NumberFormatException. T3 supplies -50 and produces IllegalArgumentException. T4 requests 9000 against a 4500 balance and produces IllegalStateException. T5 simulates a missing card-reader file and produces CardReaderException. The captured output shows finally running for all five cases.

Throwable hierarchy: Throwable branches into Error and Exception. Exception includes checked IOException and FileNotFoundException, and unchecked RuntimeException subclasses such as NumberFormatException, IllegalArgumentException and IllegalStateException. CardReaderException extends Exception.

An uncaught non-numeric input produces NumberFormatException. The top stack frame identifies Integer.parseInt, the next identifies ATM.withdraw, and the application frame identifies ATMMain.main. This sequence describes where the exception originated and how it propagated. Exact line numbers depend on compilation.

A single catch(Exception) is acceptable only at a final application boundary where unexpected failures are logged and converted to a safe response. It is not a good replacement for domain-specific handling because it hides the difference between invalid input, missing resources and programming defects.

Without exceptions, the program could use Scanner.hasNextInt(), then check amount > 0 and amount <= balance, returning explicit status codes. Both approaches are O(1) time and O(1) auxiliary space for one transaction. Creating and throwing an exception has a larger constant cost because stack-trace data is captured; status-code validation is generally cheaper for expected failures.

ATM test results

Test	Input	Observed path	finally
T1	500	Success; balance 4500	Yes
T2	abc	NumberFormatException	Yes
T3	-50	IllegalArgumentException	Yes
T4	9000	IllegalStateException	Yes
T5	card missing	CardReaderException	Yes

Q2 – User-Defined Exceptions and Propagation in an Online Examination Portal

ExamException is a checked base exception. SubmissionStorageException extends it and represents a domain-specific storage failure. Storage.save declares throws SubmissionStorageException. If the file exceeds 500 KB, the program simulates IOException and wraps it with the message 'Could not store answer02.pdf'.

Propagation path: storage detects the I/O failure, wraps it in SubmissionStorageException, and propagates the domain exception to the caller/controller. The controller catches ExamException and displays a meaningful message. The original IOException remains

accessible through getCause() for diagnostic logging.

Observed output: answer01.pdf is saved. answer02.pdf is rejected with 'Error: Could not store answer02.pdf' and 'Cause: Disk quota exceeded'. The cause type is IOException. This confirms exception chaining.

If the cause were omitted, getCause() would return null. The application would retain the high-level message but lose the underlying disk-quota explanation and original diagnostic stack trace.

Checked exceptions require callers to catch or declare them, making the recoverable failure path visible in method signatures. If these exceptions extended RuntimeException, callers would not be required to catch or declare them. That reduces signature overhead but weakens compile-time enforcement. Checked exceptions are appropriate when the caller can respond, for example by asking for a smaller file or retry.

Exception construction allocates an object and captures stack information. If call-stack depth is d, stack-trace capture and storage are proportional to O(d). Chaining retains the cause object and its trace, so memory includes both exception objects and both traces. Returning a status value is usually cheaper.

Q3 – Resource Safety, Debugging and Robust Code

The report generator opens an output writer and input readers for store files. Every opened resource must be closed even if a read or write fails.

A fragile manual-close pattern can leak a reader if an exception occurs after opening it but before the explicit close. Closing only the writer in finally does not close a separate reader. The supplementary tracked-resource test injects a failure before reader.close(); it reports one manual reader still open.

Try-with-resources declares the writer and reader in the resource header. Java closes each declared resource automatically on normal or exceptional exit. Resources close in reverse declaration order, so the reader closes before the writer. The supplementary test reports zero readers remaining open.

The original report experiment injected a failure after processing store 2 and generated partial output in both versions. This shows that automatic resource closure does not make a report transaction atomic. A production implementation should write to a temporary file and rename it only after all store files are processed successfully.

If the body throws an exception and close() also throws, the body exception remains primary and the close exception is attached as a suppressed exception, accessible through getSuppressed().

Useful debugging breakpoints are after each resource opens, after reads, before closes and inside catch/finally blocks. Logging the store filename and operation identifies the failing input.

For f store files and N total records, processing is O(N) plus file-open overhead. With one reader and one writer open at a time, maximum simultaneously open resources is O(1), independent of f. A correct manual finally approach can achieve this, but try-with-resources makes closure structurally reliable.

Q4 – Thread Creation and the Thread Life Cycle

Extending Thread puts task logic in a Thread subclass, but uses Java's single inheritance. Implementing Runnable separates the task from the execution mechanism and allows the task to be used with Thread or ExecutorService. Runnable is generally more reusable.

Thread states: NEW before start; RUNNABLE when eligible to execute; BLOCKED while waiting for a monitor; WAITING while waiting indefinitely; TIMED_WAITING during sleep or timed waits; TERMINATED after run completes. Calling run() directly is an ordinary method call on the current thread. Calling start() asks the JVM to create/schedule a new thread.

start() starts concurrent execution; sleep() pauses the current thread without releasing held monitors; join() makes the caller wait for another thread to terminate; isAlive() checks whether a started thread has not terminated.

The four stations ran ten times, with completion orders including [2, 4, 1, 3], [3, 2, 4, 1] and [4, 3, 2, 1]. The main thread joins all stations before printing the final completion message. Different orders are normal because scheduling and timing vary.

Thread priority is a scheduling hint, not a correctness guarantee. A daemon thread does not keep the JVM alive after all non-daemon threads terminate; essential work should not depend on a daemon completing.

With an ExecutorService fixed pool of p workers and n tasks, idealized task work is approximately O(n/p), excluding scheduling and contention. A fixed pool uses O(p) thread stacks; creating one thread per task may use O(n) thread stacks.

Ten observed completion orders

Run	Finish order
1	[2, 4, 1, 3]
2	[3, 2, 4, 1]

Run	Finish order
3	[3, 4, 1, 2]
4	[1, 3, 2, 4]
5	[3, 1, 4, 2]
6	[1, 3, 2, 4]
7	[3, 2, 1, 4]
8	[3, 4, 1, 2]
9	[4, 2, 3, 1]
10	[4, 3, 2, 1]

Q5 – Race Conditions in a Cinema Seat-Booking System

The critical section is the check-then-book sequence: read booked[seat], change it to true and increment bookingsMade. Without atomicity, two threads can both observe false and both increment the counter, selling the same seat twice.

The synchronized method allows one thread at a time to execute the method on the same SeatBooking object. The object's intrinsic monitor protects both the seat array and booking counter. One hundred threads compete for ten seats.

The captured execution reports 'Seats sold: 10'. Ten repeated synchronized executions also report exactly ten. The required invariant is that each seat is sold at most once.

A synchronized method locks the receiver for the method duration. A synchronized block can restrict the lock to only the critical section. Both use an intrinsic monitor. A static synchronized method locks the Class object and is appropriate for protecting shared static state.

For a cinema with 20 screens, use independent per-screen or per-seat locking so unrelated screens can proceed concurrently. A database unique constraint or transaction should enforce uniqueness across multiple application servers.

AtomicBoolean.compareAndSet can atomically claim a seat with O(s) state for s seats and O(1) expected operation cost. ReentrantLock provides explicit lock/unlock and tryLock but must be released in finally. ConcurrentHashMap.putIfAbsent can claim a seat key with expected O(1) insertion and O(s) map storage. Contention and retries increase latency.

Repeated synchronized runs

Run	Seats sold
1	10
2	10
3	10
4	10
5	10
6	10
7	10
8	10
9	10
10	10

Q6 – Deadlock and Inter-Thread Communication

Two transfers can deadlock if one thread locks wallet A then waits for B while another locks B then waits for A. The four Coffman conditions are mutual exclusion, hold-and-wait, no pre-emption and circular wait.

Assign each wallet a unique immutable ID and acquire locks in ascending ID order. All transfers then acquire locks in the same global order, eliminating circular wait. The wallet total remains conserved.

The producer-consumer counter is a circular buffer with capacity five. put waits while count is five, inserts at index in, advances in modulo five, increments count and calls notifyAll. take waits while count is zero, removes at out, advances out modulo five, decrements count and calls notifyAll.

wait() must be called while holding the same object's monitor. The condition must be checked in a while loop because wakeups may be spurious or another consumer may consume the item before the awakened thread reacquires the monitor. notify wakes one arbitrary waiter; notifyAll wakes all waiters so each can re-check its condition.

The observed total is 210, the sum of tokens 1 through 20. The wallet total after concurrent transfers is 2000. sleep pauses without releasing a monitor; wait releases the monitor and reacquires it before returning.

ArrayBlockingQueue<Integer>(5) provides a bounded thread-safe queue with blocking put/take. A ReentrantLock.tryLock with timeout can avoid indefinite transfer waits; acquired locks must be released in finally. Queue operations are O(1) for a fixed-capacity circular buffer and use O(5) storage.

Q7 – Design a Concurrent, Fault-Tolerant Processor from First Principles

The requirements are to process 10,000 score files, validate scores in 0–100, tolerate corrupt or missing files, execute independent files concurrently and produce a complete success/failure summary.

Each file is processed by a Callable. ExecutorService runs tasks in a fixed pool. Future.get collects each result. ConcurrentLinkedQueue stores results safely. Expected file and validation exceptions are caught inside each Callable and converted into failed result objects, so one bad file does not stop the batch.

The generator creates corrupt inputs at indices divisible by 100 and missing inputs at indices whose remainder is 1. This produces 100 corrupt and 100 missing files. The submitted run recorded 9,800 successes and 200 failures for each pool size. Each summary contained 10,000 results.

Measured times: pool size 1 = 605 ms; 2 = 211 ms; 4 = 211 ms; 8 = 163 ms. In this run, 2 workers improved substantially over 1, 4 did not improve over 2, and 8 was fastest. The values depend on the runtime, CPU quota, I/O cache and workload.

A pool of four or eight is a reasonable candidate for this observed workload, but production pool size should be selected using repeated measurements on the target machine. More workers can increase context switching, memory use and I/O contention.

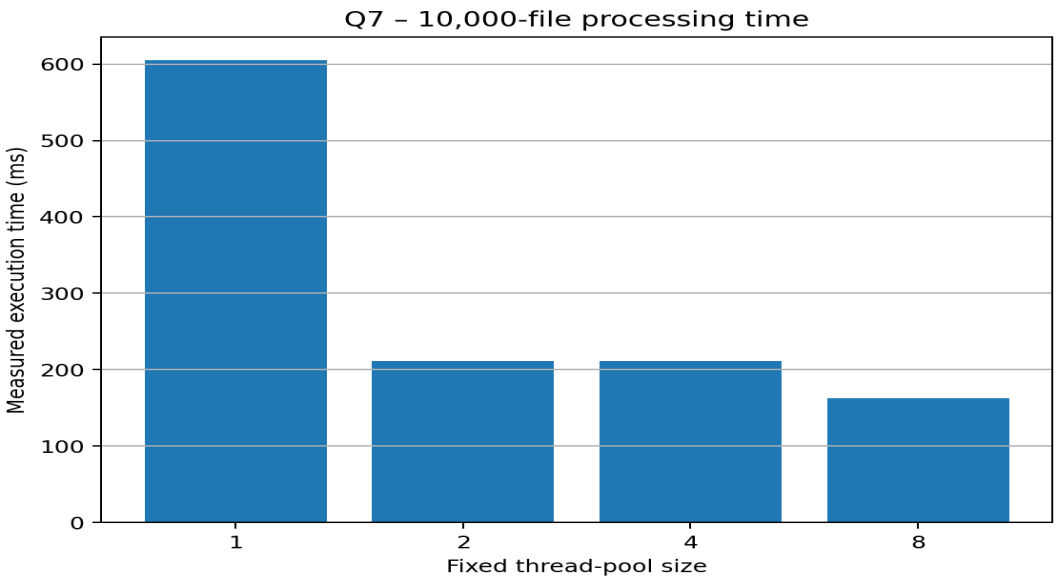
Calling get() on every Future ensures each submitted task contributes a result before summary calculation. Each Callable catches expected errors and returns a failed result. Result and Future storage is O(n) for n files; worker thread stacks are O(p) for p workers.

Parallel streams use the common ForkJoinPool by default and offer concise parallel processing, but provide less explicit control over pool sizing and failure isolation. One thread per file could create 10,000 thread stacks and exhaust resources. A fixed ExecutorService bounds worker concurrency.

For n files with bounded per-file work, total work is O(n), with idealized parallel work approximately O(n/p) for p workers. Result storage is O(n) and thread-stack storage is O(p).

Measured pool-size results

Pool	Files	Success	Failure	Time ms
1	10000	9800	200	605
2	10000	9800	200	211
4	10000	9800	200	211
8	10000	9800	200	163



Overall Conclusion

The seven tasks demonstrate exception classification and propagation, cause-preserving exception chaining, automatic resource management, thread lifecycle control, atomic updates to shared state, deadlock prevention, producer-consumer coordination and fault-tolerant concurrent execution.

The central design principles are to handle exceptions at the appropriate layer, preserve causes when translating exceptions, use try-with-resources for resource ownership, protect shared mutable state with a clear lock, acquire multiple locks in a consistent order, re-check wait conditions in loops and use a bounded executor for large independent workloads.

Evidence inventory

Task folders contain Java source code and supporting evidence. logs/ contains execution output; data/ contains raw CSV files; screenshots/ contains rendered console-output evidence panels; graphs/ contains performance plots.